



ISO 9001: 2015
SOCOTEC SCP000722Q

REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay



IT112 – WEB SYSTEMS TECHNOLOGIES

BSIT-2B 2nd Semester

Campus Service Hub:

A Student Guide and Walkthrough in Bicol University Polangui

Group Name:

Code: <breakers>

Members:

Carilla, Grace Ann S.

Lamud, Ella Mae C.

Olarcos, Joshua L.

Requio, Rein Ashley S.

Rubio, Ma. Miscy M.

Sentillas, Tiffany C.

Professor

Guillermo V. Red, Jr

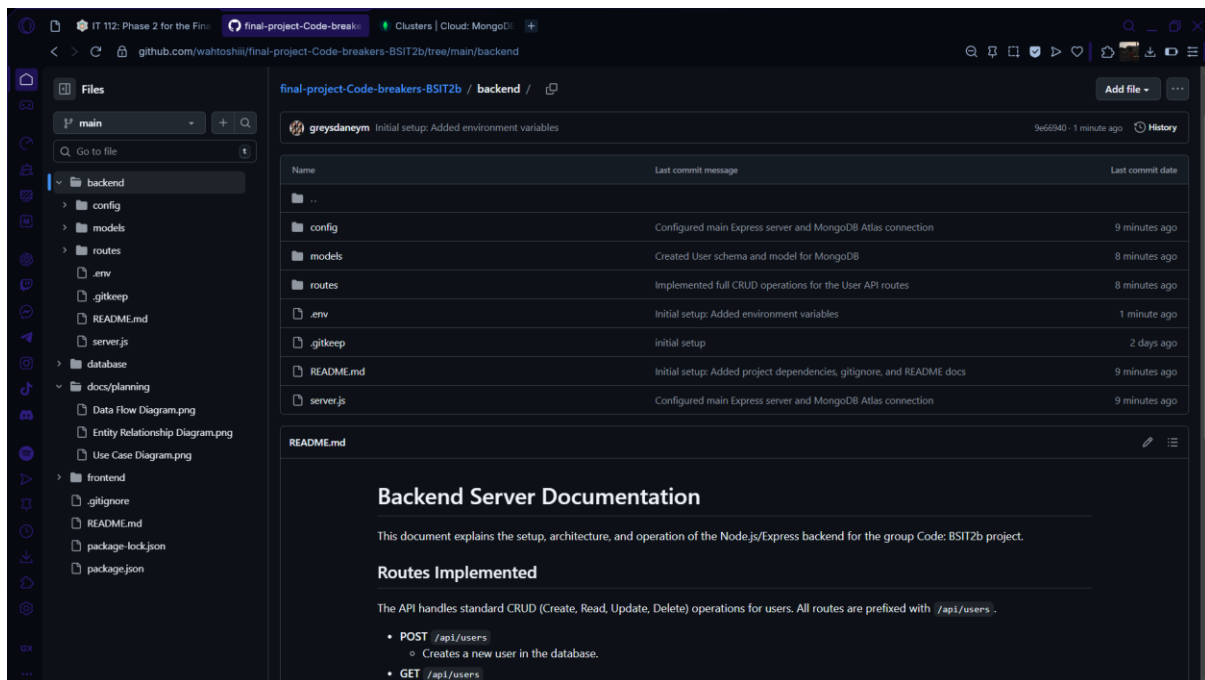
April 2026



Overview of Backend Setup

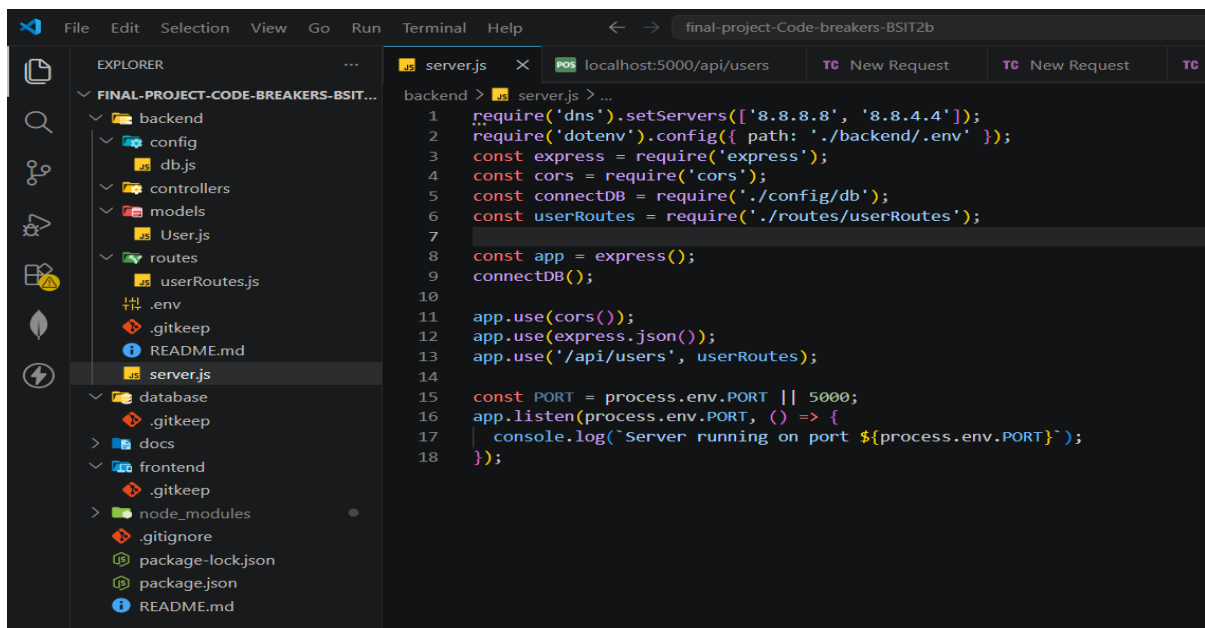
Node.js and Express were used to develop our backend. It deals with user data, request processing, and database interaction in storing and retrieving data.

Code Snippets



The screenshot shows a GitHub repository for 'final-project-Code-breakers-BSIT2b' with a focus on the 'backend' directory. The repository structure includes files like .env, .gitkeep, README.md, server.js, and folders for config, models, routes, database, docs/planning, and frontend. The README.md file is open, displaying 'Backend Server Documentation' which explains the setup and lists implemented routes. The routes section specifies that the API handles standard CRUD operations for users, with all routes prefixed with '/api/users'. The implemented routes are:

- POST /api/users: Creates a new user in the database.
- GET /api/users



The screenshot shows a code editor with the 'server.js' file open. The code defines an Express application, sets up CORS, connects to a database, and implements a route for creating a new user. The code is as follows:

```
1 require('dns').setServers(['8.8.8.8', '8.8.4.4']);
2 require('dotenv').config({ path: './backend/.env' });
3 const express = require('express');
4 const cors = require('cors');
5 const connectDB = require('./config/db');
6 const userRoutes = require('./routes/userRoutes');
7
8 const app = express();
9 connectDB();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use('/api/users', userRoutes);
14
15 const PORT = process.env.PORT || 5000;
16 app.listen(process.env.PORT, () => {
17   console.log(`Server running on port ${process.env.PORT}`);
18 });
```



ISO 9001: 2015
SOCOTEC SCP000722Q

REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay



```
File Edit Selection View Go Run Terminal Help  
final-project-Code-breakers-BSIT2b  
EXPLORER  
FINAL-PROJECT-C...  
  backend  
  config  
  db.js  
  controllers  
  models  
  User.js  
  routes  
  userRoutes.js  
  .env  
  .gitkeep  
  README.md  
  server.js  
  database  
  .gitkeep  
  docs  
  frontend  
  .gitkeep  
  node_modules  
  .gitignore  
  package-lock.json  
  package.json  
  README.md  
New Request TC New Request  
README.md backend .gitignore  
backend > models > User.js > ...  
1 const mongoose = require('mongoose');  
2  
3 const userSchema = new mongoose.Schema({  
4   name: String,  
5   email: String,  
6   password: String,  
7   role: String  
8 });  
9  
10 module.exports = mongoose.model('User', userSchema);
```

```
File Edit Selection View Go Run Terminal Help  
final-project-Code-breakers-BSIT2b  
userRoutes.js  
1 const express = require('express');  
2 const router = express.Router();  
3 const User = require('../models/User');  
4  
5 // 1. Create a user (POST)  
6 router.post('/', async (req, res) => {  
7   try {  
8     const user = new User(req.body);  
9     await user.save();  
10    res.status(201).json(user);  
11  } catch (err) {  
12    res.status(400).json({ message: err.message });  
13  }  
14 });  
15  
16 // 2. Read all users (GET)  
17 router.get('/', async (req, res) => {  
18   try {  
19     const users = await User.find();  
20     res.json(users);  
21   } catch (err) {  
22     res.status(500).json({ message: err.message });  
23   }  
24 });  
25  
26 // 3. Update a user (PUT)  
27 router.put('/:id', async (req, res) => {  
28   try {  
29     // The '{ new: true }' option tells Mongoose to return the updated user, not the old one.  
30     const updatedUser = await User.findByIdAndUpdate(  
31       req.params.id,  
32       req.body,  
33       { new: true }  
34     );  
35  
36     if (!updatedUser) {  
37       return res.status(404).json({ message: 'User not found' });  
38     }  
39     res.json(updatedUser);  
40   } catch (err) {  
41     res.status(400).json({ message: err.message });  
42   }  
43 });  
44  
45 // 4. Delete a user (DELETE)  
46 router.delete('/:id', async (req, res) => {  
47   try {  
48     const deletedUser = await User.findByIdAndDelete(req.params.id);  
49  
50     if (!deletedUser) {  
51       return res.status(404).json({ message: 'User not found' });  
52     }  
53     res.json({ message: 'User deleted successfully' });  
54   } catch (err) {  
55     res.status(500).json({ message: err.message });  
56   }  
57 }  
58 }  
59 module.exports = router;
```

API Testing Screenshots

THUNDER CLIENT

New Request

POST http://localhost:5000/api/users

Send

Status: 201 Created Size: 142 Bytes Time: 396 ms

Response

```
1 {  
2   "name": "Juan Tamad",  
3   "email": "juantamad@gmail.com",  
4   "password": "akosijuan23",  
5   "role": "project manager",  
6   "_id": "69de5d92ba3a384cd82acbee",  
7   "__v": 0  
8 }
```

THUNDER CLIENT

New Request

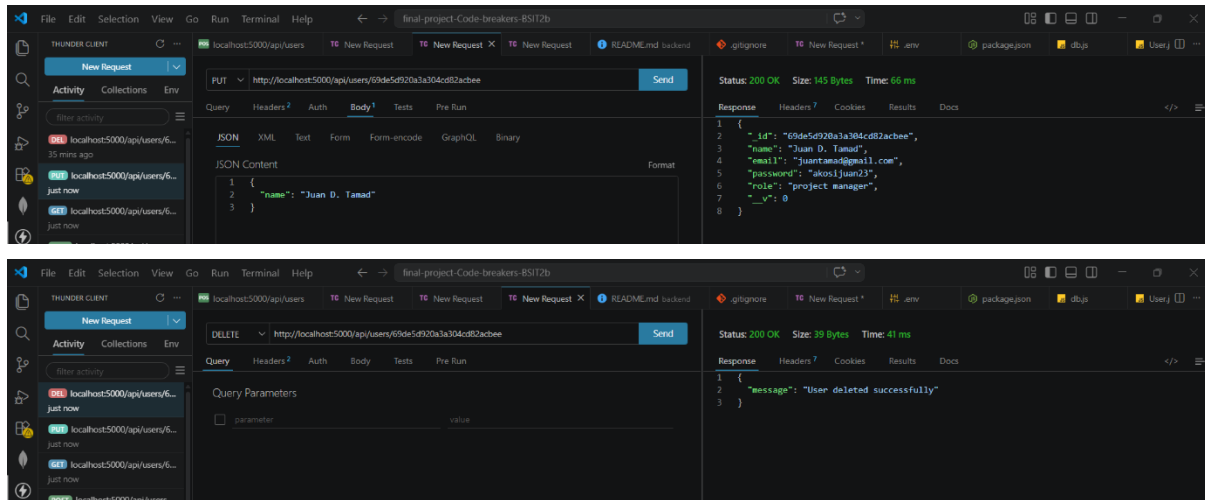
GET http://localhost:5000/api/users

Send

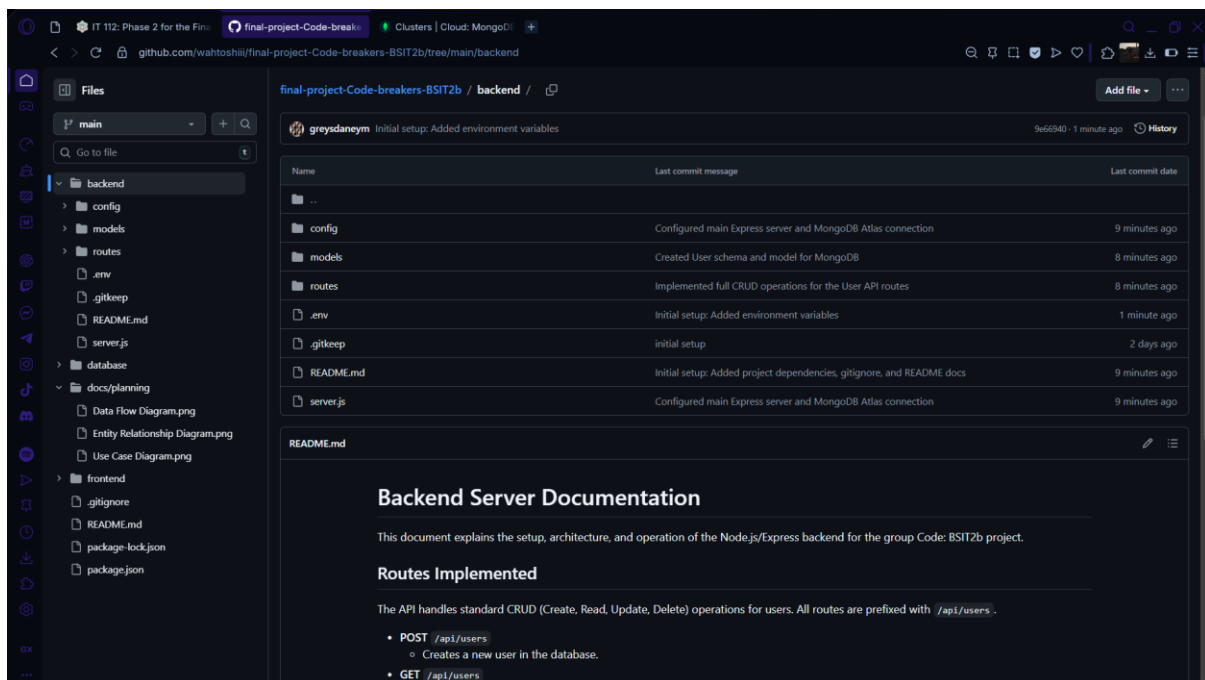
Status: 200 OK Size: 144 Bytes Time: 57 ms

Response

```
1 {  
2   "_id": "69de5d92ba3a384cd82acbee",  
3   "name": "Juan Tamad",  
4   "email": "juantamad@gmail.com",  
5   "password": "akosijuan23",  
6   "role": "project manager",  
7   "__v": 0  
8 }  
9  
10 ]
```



GitHub Commits



Contribution of Each Members

Carilla, Grace Ann S. - As the Backend Developer, I took the lead in establishing the core server-side architecture. I was responsible for developing the Node.js POST routes, configuring the secure connection to MongoDB Atlas, and implementing essential middleware such as CORS to ensure the frontend forms could successfully communicate with the backend API and store user data.



Lamud, Ella Mae C. - As the GitHub Manager, my main contribution was to make sure that all backend code was properly organized, committed, and easy to understand for the group. I ensured that all backend files were placed inside the /backend folder to maintain a clean and consistent folder structure. I also made sure that every commit had clear and specific messages, such as “Added user model and routes,” so that anyone in the group can easily track what changes were made. In addition, I created a README.md inside the /backend folder where I explained the routes implemented, current models, database connection setup, and the steps on how to run the server locally.

Olarcos, Joshua L. - As the Project Manager, I oversaw the alignment of the frontend development with the project's established ERD and Use Case diagrams. During the backend integration phase, I managed the team's technical workflow and actively assisted the backend developer by troubleshooting form submission errors, debugging fetch() requests, and ensuring that data validation issues were resolved for successful database insertion.

Requio, Rein Ashley S. - As our group's tester, I made sure everything worked properly from start to finish. I used Postman to check our APIs and confirm they returned the right data and responses. I also documented the tests with clear screenshots and reported any issues so they could be fixed quickly. Overall, I helped ensure our project was fully working and ready to submit.

Rubio, Ma. Miscoy M. – In my role as the database manager, my contribution to the group involved setting up MongoDB. I created an active IP address and configured the MongoDB connection string. For security purposes, the connection string was stored in the .env file in VS Code rather than being exposed in the source code.

Sentillas, Tiffany C. - As a frontend developer, I did not have any direct involvement in the process of developing the backend part of the website. My main role was to compile the lab report and ensure that all the necessary parts of the lab report were in place.